

**NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY
GREATER NOIDA**



**Problem Solving Approaches
(BCSCC0452)**

**Name of student: Aditya Raj
Roll No: 2401331730005**

**Submitted to:
Mr. Manoj Prajapati
Assistance Professor, Training Department**

COURSE: B.TECH.

SEMESTER: IV

Department of Electronics Engineering(VLSI)

Assignment : 1

Question : 1

You are given a 3-digit number n, Find whether it is an Armstrong number or not.

Code:

```
#include <iostream>
#include <cmath>

bool isArmstrong(int n) {
    int original = n;
    int sum = 0;
    while (n > 0) {
        int digit = n % 10;
        sum += (digit * digit * digit);
        n /= 10;
    }
    return sum == original;
}

int main() {
    int n;
    std::cout << "Enter a 3-digit number: ";
    std::cin >> n;
    if (isArmstrong(n)) std::cout << n << " is an Armstrong number." << std::endl;
    else std::cout << n << " is not an Armstrong number." << std::endl;
    return 0;
}
```

Output:

```
Enter a 3-digit number: 371
371 is an Armstrong number.
```

Question : 2

Given a number n, determine whether it is a prime number or not. Note: A prime number is a number greater than 1 that has no positive divisors other than 1 and itself.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int n, i;
    cin >> n;

    if (n <= 1) {
        cout << "Not Prime";
        return 0;
    }

    for (i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            cout << "Not Prime";
            return 0;
        }
    }

    cout << "Prime";
    return 0;
}
```

Output:

```
79
Prime
```

Question : 3

Given an array arr[] of N positive integers. The task is to find the sum of all palindrome numbers present in the array. Print the total sum.

Code:

```
#include <iostream>
using namespace std;

bool isPalindrome(int n) {
    int rev = 0, temp = n;
    while (temp > 0) {
        rev = rev * 10 + temp % 10;
        temp /= 10;
    }
    return rev == n;
}

int main() {
    int n, arr[100], sum = 0;

    cout << "Enter size: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    for (int i = 0; i < n; i++) {
        if (isPalindrome(arr[i]))
            sum += arr[i];
    }

    cout << "Sum = " << sum;
}
```

Output:

```
Enter size: 5
Enter elements: 121 123 11 22 10
Sum = 154
```

Question : 4

Find the highest power of 2 less than or equal to a given number.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter number: ";
    cin >> n;

    int power = 1;
    while (power * 2 <= n)
        power *= 2;

    cout << "Result = " << power;
}
```

Output:

```
Enter number: 20
Result = 16
```

Question : 5

Generate a series where each term is the factorial of the position.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter n: ";
    cin >> n;

    int fact = 1;
    for (int i = 1; i <= n; i++) {
        fact *= i;
        cout << fact << " ";
    }
}
```

Output:

```
Enter n: 5
1 2 6 24 120
```

Question : 6

Write a program to count the number of vowels in a given string.

Code:

```
#include <iostream>
using namespace std;

int main() {
    string str;
    cout << "Enter string: ";
    getline(cin, str);

    int count = 0;
    for (char c : str) {
        c = tolower(c);
        if (c=='a' || c=='e' || c=='i' || c=='o' || c=='u')
            count++;
    }

    cout << "Vowels = " << count;
}
```

Output:

```
Enter string: hello world
Vowels = 3
```

Question : 7

Draw the following Patterns for N=5

Code:

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++)
            cout << "*";
        cout << endl;
    }
}
```

Output:

```
*
**
***
****
*****
```


Question : 8

Write a program that takes an integer n as input and prints the multiplication table of n from n * 1 to n * 10 using Recursion.

Code:

```
#include <iostream>
using namespace std;

void table(int n, int i) {
    if (i > 10) return;

    cout << n << " * " << i << " = " << n * i << endl;
    table(n, i + 1);
}

int main() {
    int n;
    cout << "Enter number: ";
    cin >> n;

    table(n, 1);
}
```

Output:

```
Enter number: 5
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
```

Question : 9

Write a program to calculate the sum of all integers from 1 to a given number N. The program should take N as input and output the total sum using recursion.

Code:

```
#include <iostream>
using namespace std;

int sum(int n) {
    if (n == 1) return 1;
    return n + sum(n - 1);
}

int main() {
    int n;
    cout << "Enter n: ";
    cin >> n;

    cout << "Sum = " << sum(n);
}
```

Output:

```
Enter n: 5
Sum = 15
```

Question : 10

Write a recursive function to calculate the Greatest Common Divisor (GCD) of two numbers using Euclid's algorithm. The function should take two integers as input and return their GCD.

Code:

```
#include <iostream>
using namespace std;

int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

int main() {
    int a, b;
    cout << "Enter two numbers: ";
    cin >> a >> b;

    cout << "GCD = " << gcd(a, b);
}
```

Output:

```
Enter two numbers: 48 18
GCD = 6
```

Question : 11

Write a program to implement Binary Search using recursion.

Code:

```
#include <iostream>
using namespace std;

int binarySearch(int arr[], int l, int r, int key) {
    if (l > r) return -1;

    int mid = (l + r) / 2;

    if (arr[mid] == key) return mid;
    else if (arr[mid] > key)
        return binarySearch(arr, l, mid - 1, key);
    else
        return binarySearch(arr, mid + 1, r, key);
}

int main() {
    int n, key;
    cout << "Enter size: ";
    cin >> n;

    int arr[n];
    cout << "Enter sorted array: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Enter key: ";
    cin >> key;

    int result = binarySearch(arr, 0, n - 1, key);

    if (result != -1)
        cout << "Found at index " << result;
    else
        cout << "Not found";
}
```

Output:

```
Enter size: 5
Enter sorted array: 1 3 5 7 9
Enter key: 5
Found at index 2
```

Question : 12

Given an array nums sorted in non-decreasing order, return the maximum between the number of positive integers and the number of negative integers. In other words, if the number of positive integers n nums is pos and the number of negative integers is neg, then return the maximum of pos and neg. Note that 0 is neither positive nor negative.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int n, pos = 0, neg = 0;

    cout << "Enter size: ";
    cin >> n;

    int arr[n];
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
        if (arr[i] > 0) pos++;
        else if (arr[i] < 0) neg++;
    }

    cout << "Result = " << max(pos, neg);
}
```

Output:

```
Enter size: 6
Enter elements: -2 -1 -1 1 2 3
Result = 3
```

Question : 13

Write a program to sort a list of elements using the Merge Sort algorithm.

Code:

```
#include <iostream>
using namespace std;

void merge(int arr[], int l, int m, int r) {
    int i = l, j = m + 1, k = 0;
    int temp[100];

    while (i <= m && j <= r) {
        if (arr[i] < arr[j]) temp[k++] = arr[i++];
        else temp[k++] = arr[j++];
    }
    while (i <= m) temp[k++] = arr[i++];
    while (j <= r) temp[k++] = arr[j++];

    for (int i = l; i <= r; i++)
        arr[i] = temp[i - l];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    int n;
    cout << "Enter size: ";
    cin >> n;

    int arr[n];
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    mergeSort(arr, 0, n - 1);

    cout << "Sorted: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
}
```

Output:

```
Enter size: 5
Enter elements: 5 2 9 1 3
Sorted: 1 2 3 5 9
```

Question : 14

Given an array of positive integers arr[] of size n, the task is to find second largest distinct element in the array.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter size: ";
    cin >> n;

    int arr[n];
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    int first = -1, second = -1;

    for (int i = 0; i < n; i++) {
        if (arr[i] > first) {
            second = first;
            first = arr[i];
        } else if (arr[i] > second && arr[i] != first) {
            second = arr[i];
        }
    }

    cout << "Second Largest = " << second;
}
```

Output:

```
Enter size: 6
Enter elements: 12 35 1 10 34 1
Second Largest = 34
```


Question : 15

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target. You may assume that each input would have exactly one solution, and you may not use the same element twice. You can return the answer in any order.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int n, target;
    cout << "Enter size: ";
    cin >> n;

    int arr[n];
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Enter target: ";
    cin >> target;

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[i] + arr[j] == target) {
                cout << "Indices: " << i << " " << j;
                return 0;
            }
        }
    }
}
```

Output:

```
Enter size: 4
Enter elements: 2 7 11 15
Enter target: 9
Indices: 0 1
```


Question : 16

Given a 1-indexed array of integers numbers that is already sorted in non-decreasing order, find two numbers such that they add up to a specific target number. Let these two numbers be numbers[index1] and numbers[index2] where $1 \leq \text{index1} < \text{index2} \leq \text{numbers.length}$.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int n, target;
    cout << "Enter size: ";
    cin >> n;

    int arr[n];
    cout << "Enter sorted array: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Enter target: ";
    cin >> target;

    int l = 0, r = n - 1;

    while (l < r) {
        if (arr[l] + arr[r] == target) {
            cout << "Indices: " << l + 1 << " " << r + 1;
            return 0;
        } else if (arr[l] + arr[r] < target)
            l++;
        else
            r--;
    }
}
```

Output:

```
Enter size: 4
Enter sorted array: 2 7 11 15
Enter target: 9
Indices: 1 2
```

Question : 17

Reverse an array arr[]. Reversing an array means rearranging the elements such that the first element becomes the last, the second element becomes second last and so on.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter size: ";
    cin >> n;

    int arr[n];
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    for (int i = 0; i < n / 2; i++)
        swap(arr[i], arr[n - i - 1]);

    cout << "Reversed: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
}
```

Output:

```
Enter size: 6
Enter elements: 1 4 3 2 6 5
Reversed: 5 6 2 3 4 1
```

Question : 18

Given an array `arr[]` of size `n` containing distinct integers within the range `[1, n+2]`, find the two missing numbers from the first `n+2` natural numbers. Constraints: The solution must run in $O(N)$ time and use $O(1)$ extra space. The array does not contain duplicate values.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter size: ";
    cin >> n;

    int arr[n];
    int total = (n + 2) * (n + 3) / 2;
    int sum = 0;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
        sum += arr[i];
    }

    int diff = total - sum;
    int pivot = diff / 2;

    int sum1 = 0;
    int total1 = pivot * (pivot + 1) / 2;

    for (int i = 0; i < n; i++) {
        if (arr[i] <= pivot)
            sum1 += arr[i];
    }

    int x = total1 - sum1;
    int y = diff - x;

    cout << "Missing numbers: " << x << " " << y;
```

Output:

```
Enter size: 6
Enter elements: 1 2 4 6 3 8
Missing numbers: 5 7
```

Question : 19

Trapping Rain Water: Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining. Input: height = [0,1,0,2,1,0,1,3,2,1,2,1] Output: 6

Code:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter size: ";
    cin >> n;

    int arr[n];
    cout << "Enter heights: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    int left[n], right[n];

    left[0] = arr[0];
    for (int i = 1; i < n; i++)
        left[i] = max(left[i - 1], arr[i]);

    right[n - 1] = arr[n - 1];
    for (int i = n - 2; i >= 0; i--)
        right[i] = max(right[i + 1], arr[i]);

    int water = 0;
    for (int i = 0; i < n; i++)
        water += min(left[i], right[i]) - arr[i];

    cout << "Water = " << water;
}
```

Output:

```
Enter size: 12
Enter heights: 0 1 0 2 1 0 1 3 2 1 2 1
Water = 6
```

Question : 20

Container With Most Water: You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the ith line are (i, 0) and (i, height[i]). Find two lines that together with the x-axis form a container, such that the container contains the most water. Return the maximum amount of water a container can store.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter size: ";
    cin >> n;

    int arr[n];
    cout << "Enter heights: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    int l = 0, r = n - 1;
    int maxArea = 0;

    while (l < r) {
        int height = min(arr[l], arr[r]);
        int width = r - l;
        maxArea = max(maxArea, height * width);

        if (arr[l] < arr[r])
            l++;
        else
            r--;
    }

    cout << "Max Area = " << maxArea;
}
```

Output:

```
Enter size: 9
Enter heights: 1 8 6 2 5 4 8 3 7
Max Area = 49
```